

Lecture 6 : ADC & DAC

18 조 – 이해리, 장동화

I. INTRODUCTION

A. 실험 목적

이번 실험을 통해 ADC(Analog to Digital Converter)와 DAC(Digital to Analog Converter)의 작동원리를 알아보고 간단한 신호를 디지털/아날로그 양방향으로 변환해본다.

ADC를 통해 현실세계의 아날로그 물리량을 디지털 신호로 번역할 수 있다. 이러한 디지털 신호는 정보처리, 연산, 데이터 변환 그리고 제어 시스템에 사용될 수 있다. DAC는 저장된 데이터 또는 디지털 처리의 결과를 제어, 정보 출력 또는 아날로그 처리를 위해 다시 현실세계로 변환한다.

II. EXPERIMENTAL RESULTS

A. 실험 1 – ADC / DAC 상호 변환

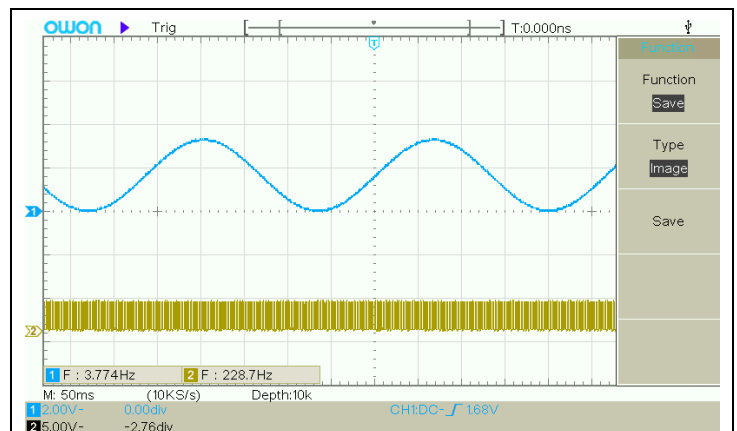
첫번째 실험에서, ADC와 DAC의 상호 변환과정을 구현해보았다. DAC로 정현파를 출력하여 이를 ADC로 다시 읽어서 확인해보았고, 이어서 함수발생기로 생성한 정현파를 ADC로 읽어서 그 값을 그대로 DAC로 출력해서 확인해보았다.

실행한 코드는 다음과 같다.

```
462 /* USER CODE BEGIN 4 */
463
464 /* ADC end-of-conversion interrupt callback */
465 void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef* hadc)
466 {
467     if (hadc->Instance == ADC1)
468     {
469         g_adc_last = (uint16_t)HAL_ADC_GetValue(hadc);
470     }
471 }
472
473 void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim){
474     if(htim->Instance == TIM1){
475         PWM_Counter++;
476         if (PWM_Counter >= 160) {
477             DAC_Counter++;
478             dac_value = (uint16_t)(2010 + 2000 * sinf(2.0f * 3.14159f * ((float)DAC_Counter) / 160.0f));
479             HAL_DAC_SetValue(&hdac1, Channel: DAC_CHANNEL_2, Alignment: DAC_ALIGN_12B_R, Data: dac_value);
480
481             if(enable_printf){
482                 HAL_ADC_Start_IT(&hadc1);
483                 printf("ADC=%u\r\n", (unsigned)g_adc_last);
484             }
485         }
486     }
487 }
488
489 void HAL_GPIO_EXTI_Rising_Callback(uint16_t GPIO_Pin){
490     if (GPIO_Pin == C13_SWITCH_Pin) {
491         PC13_Counter++;
492         /* Increase PWM frequency by 10 Hz each press (approximate achievable) */
493         g_tim1_pwm_freq_hz += 1000;
494         TIM1_SetFrequencyHz(freq_hz: g_tim1_pwm_freq_hz);
495
496         // enable_printf = !enable_printf;
497     }
498 }
499 /* Add helper functions for PWM frequency handling */
500 static uint32_t TIM1_GetTimerClockHz(void)
501 {
502 }
503
504 static void TIM1_SetFrequencyHz(uint32_t freq_hz)
505 {
506 }
507
508 /* USER CODE END 4 */
```

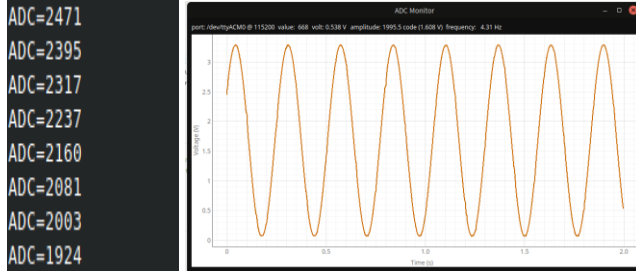
인터럽트 우선순위를 버튼, ADC, 타이머 순서로 설정해주었다. PWM 신호를 생성하여 인터럽트를 발생시켰고 일정 주기에 한 번 ADC 인터럽트, DAC 설정자를 호출하여 DAC로 생성한 아날로그 정현파 신호를 ADC를 통해 읽었다.

다음은 DAC가 생성한 아날로그 신호를 오실로스코프로 측정한 것이다.



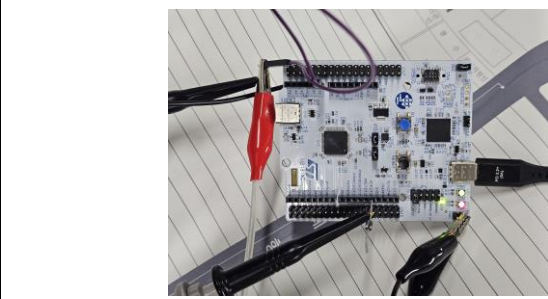
Yellow, PWM Sig / Blue, DAC Output Sig

ADC로 읽은 값을 표준출력을 통해 출력하였다. ST-LINK를 통해 생성된 가상 COM 통신으로 PC에서 표준입출력이 가능하다. 다음은 표준입출력을 터미널에서 직접 본 것과 파이썬을 통해 시각화해본 것이다.

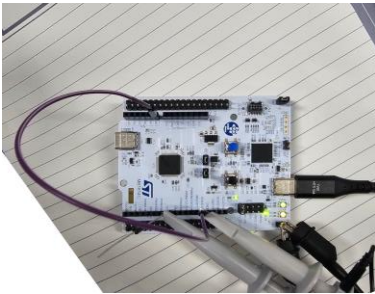


Yellow, Function Gen Output Sig / Blue, DAC Output Sig

다음은 함수발생기와 ADC 그리고 오실로스코프와 DAC를 연결한 사진이다.



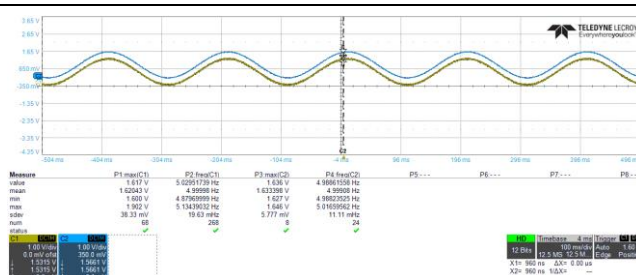
다음은 DAC, ADC 그리고 오실로스코프를 연결한 사진이다.



함수발생기로 생성된 정현파를 ADC로 읽어서 DAC로 출력하는 코드이다.

```
463 /* USER CODE BEGIN 4 */
464
465 /* ADC end-of-conversion interrupt callback */
466 void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef* hadc)
467 {
468     if (hadc->Instance == ADC1)
469     {
470         g_adc_last = (uint16_t)HAL_ADC_GetValue(hadc);
471         HAL_DAC_SetValue(&hdac1, Channel: DAC_CHANNEL_2, Alignment: DAC_ALIGN_12B_R, Data: g_adc_last);
472     }
473 }
474
475 void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef* htim)
476 {
477     if(htim->Instance == TIM1)
478     {
479         HAL_ADC_Start_IT(&hadc1);
480         PWM_Counter++;
481         // if (PWM_Counter >= 160) {
482     }
483 }
```

다음은 함수발생기와 DAC가 생성한 아날로그 신호를 오실로스코프로 측정한 것이다.



B. 실험 2 – NTC를 이용한 온도 측정

두번째 실험에서는 F302 보드의 NTC에서 측정한 온도 값을 F103 보드의 ADC channel에서 입력 받아 2.4ms 동안 측정된 100개의 온도 값의 평균을 실시간으로 출력해보았다. 작성한 코드는 다음과 같다.

```
uint32_t Read_ADC_Average(void)
{
    uint64_t sum = 0;
    const int samples = 100;

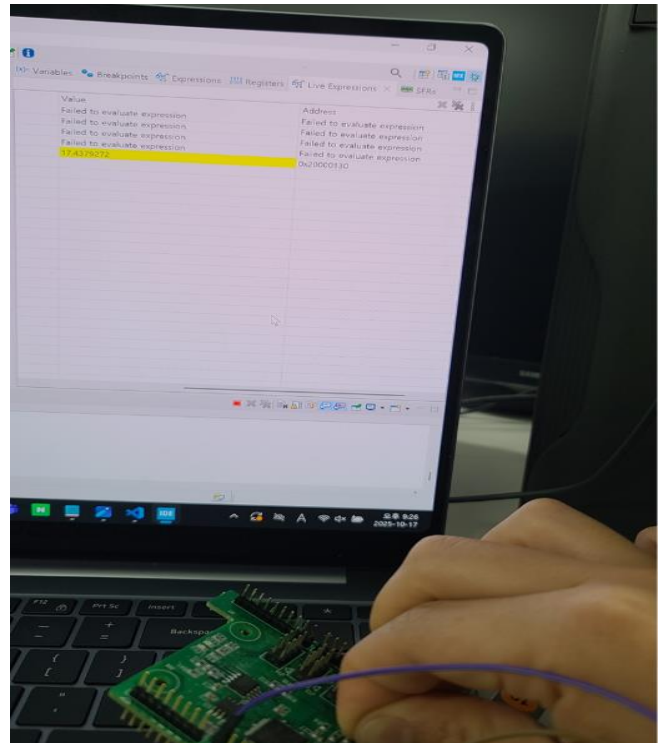
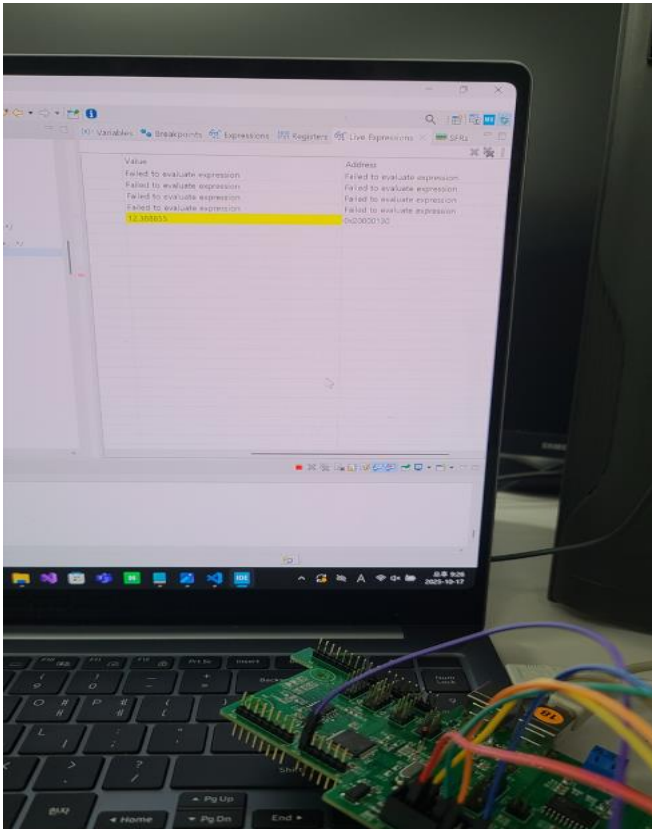
    for (int i = 0; i < samples; i++)
    {
        HAL_ADC_Start(&hadc1);
        HAL_ADC_PollForConversion(&hadc1, HAL_MAX_DELAY);
        uint32_t value = HAL_ADC_GetValue(&hadc1);
        HAL_ADC_Stop(&hadc1);
        sum += value;

        HAL_Delay(2); // 약 2.4ms 가4 (HAL Delay 해상도: 1ms)
    }

    return (uint32_t)(sum / samples);
}

float Read_NTC_Temperature(void)
{
    uint32_t avgADC = Read_ADC_Average();
    float vout = (VCC * avgADC) / 4095.0f;
    float rNTC = (R_FIXED * vout) / (VCC - vout);
    float tempK = 1.0f / ((1.0f / T0) + (1.0f / BETA_COEFF) * logf(rNTC / R0));
    return tempK - 273.15f;
}
```

위 코드의 동작을 설명하면, 우선 HAL_ADC_Start(); 함수를 통해 ADC 변환을 시작하고, HAL_ADC_GetValue(); 함수로 변환된 ADC 값을 읽는다. 그리고 읽은 샘플 값의 누적 값을 계산한 뒤에 return (uint32_t)(sum/samples); 함수를 통해 누적 값(총 합)을 샘플 수로 나누어서 평균값을 반환한다. Const int samples = 100; 에서 샘플 수가 100개로 지정되어 있기 때문에 2.4ms 동안 측정된 100개의 온도 값의 평균이 도출되는 것이다. 코드를 실행 결과는 다음과 같다.



결과 사진을 보면, 코드를 실행한 직후, F302 보드의 NTC에서 측정한 온도 값의 평균은 약 12°C 인 것을 확인할 수 있고, NTC 온도 센서 부분을 손으로 일정 시간 동안 잡고 있었을 때, 온도 값의 평균이 약 17°C까지 증가하여 측정되는 것을 확인할 수 있었다.

III. CONCLUSION

이번 실험을 통해 ADC와 DAC의 작동 원리와 사용 분야 그리고 사용 방법을 알게 됐다. 현실세계의 아날로그 신호를 ADC로 읽어 DAC로 출력해보았고 계산된 디지털 신호에서 아날로그 신호를 생성하여 다시 디지털로 변환해보았다.

REFERENCES

- [1] Walt Kester, Dan Sheingold, James Bryant, Data Conversion Handbook, Elsevier/Analog Devices, Chapter 2 "Fundamentals of Sampled Data Systems," Section 2.1 "Coding and Quantizing," pp. 2.1–2.3, 2005.